Numerieke benadering met toepassing in datawetenschappen Practicum 1

Numerieke Benadering met Toepassing in Datawetenschappen

Stefan Fabianek r0935346
Arne Van dycke r1007271
Bachelor in de ingenieurswetenschappen
Academiejaar 2025-2026

1 Bivariate veeltermbenadering **1**
Opgave 1 1
Opgave 2 2
Opgave 4 3
Opgave 5 3
Opgave 6 3
Opgave 7 4

2 Kleinstekwadratenbenadering met splinefuncties **6**
Opgave 8 6
Opgave 9 6
Opgave 10 7
Opgave 11 8
Opgave 12 8

In dit deel wordt de kleinstekwadratenbenadering van roosterdata met bivariate Legendreveeltermen uitgewerkt. De formulering in matrixvorm en de numerieke implementatie worden behandeld in opgaven 1 tot en met 7.

Opgave 1

Beschouw de benaderende functie

$$z(x,y) = \sum_{k=0}^{n} \sum_{l=0}^{m} c_{kl}\, p_k(x) q_l(y),$$

met $p_k(x) = P_k(x)$ en $q_l(y) = P_l(y)$ de Legendrebasisfuncties in respectievelijk x- en y-richting. Voor roosterpunten (x_i, y_j) wordt dan

$$z(x_i, y_j) = \sum_{k=0}^{n} \sum_{l=0}^{m} c_{kl}\, p_k(x_i) q_l(y_j).$$

Definieer de matrices $A \in \mathbb{R}^{N\times(n+1)}$ en $B \in \mathbb{R}^{M\times(m+1)}$ met elementen

$$A_{i,k+1} = p_k(x_i), \quad i = 1,\ldots,N,\ k = 0,\ldots,n,$$

$$B_{j,l+1} = q_l(y_j), \quad j = 1,\ldots,M,\ l = 0,\ldots,m.$$

Neem daarnaast $C \in \mathbb{R}^{(m+1)\times(n+1)}$ als de coëfficiëntenmatrix met $C_{l+1,k+1} = c_{kl}$. Dan geldt elementgewijs

$$(BCA^\top)_{j,i} = \sum_{l=0}^{m} \sum_{k=0}^{n} c_{kl}\, q_l(y_j) p_k(x_i) = z(x_i, y_j).$$

Dus de benaderde roosterwaarden worden gegeven door de matrix $Z = BCA^\top$.

Met $F \in \mathbb{R}^{M\times N}$ de datamatrix en $f = \mathrm{vec}(F)$ volgt voor de oorspronkelijke doelfunctie van het benaderingsprobleem:

$$\sum_{i=1}^{N} \sum_{j=1}^{M} \big(f_{ij} - z(x_i, y_j)\big)^2 = \|F - Z\|_F^2 = \|\mathrm{vec}(F) - \mathrm{vec}(Z)\|_2^2.$$

Gebruik nu de identiteit

$$\mathrm{vec}(BCA^\top) = (A \otimes B)\,\mathrm{vec}(C).$$

Definieer $c = \mathrm{vec}(C)$ en $\Gamma = A \otimes B$. Dan wordt

$$\|\mathrm{vec}(F) - \mathrm{vec}(Z)\|_2^2 = \|f - \Gamma c\|_2^2 = \|\Gamma c - f\|_2^2.$$

Hieruit volgt rechtstreeks dat

$$\arg\min_{\{c_{kl}\}} \sum_{i=1}^{N} \sum_{j=1}^{M} \big(f_{ij} - z(x_i, y_j)\big)^2 \quad \Longleftrightarrow \quad \arg\min_{c} \|\Gamma c - f\|_2^2.$$

Hiermee is het benaderingsprobleem equivalent met het minimalisatieprobleem.

Opgave 2

Uit Opgave 1 volgt dat het benaderingsprobleem herschreven kan worden als

$$\arg\min_c \|\Gamma c - f\|_2^2, \qquad \Gamma = A \otimes B, \qquad f = \mathrm{vec}(F).$$

Een kleinstekwadratenoplossing met minimale norm wordt gegeven door de pseudo-inverse:

$$c = \Gamma^+ f = (A \otimes B)^+ \,\mathrm{vec}(F).$$

De gevraagde vorm volgt zodra aangetoond is dat

$$(A \otimes B)^+ = A^+ \otimes B^+.$$

Een compacte verantwoording gebruik makend van de SVD:

$$A = U_A \Sigma_A V_A^\top, \qquad B = U_B \Sigma_B V_B^\top,$$

waarbij U en V orthogonale matrices zijn en Σ een diagonaalmatrix met de singuliere waarden. Dan geldt

$$A \otimes B = (U_A \Sigma_A V_A^\top) \otimes (U_B \Sigma_B V_B^\top).$$

Door gebruik te maken van de mixed-product eigenschap van kroneckerproducten, namelijk $(XZ) \otimes (YW) = (X \otimes Y)(Z \otimes W)$, kan dit herschreven worden als:

$$A \otimes B = (U_A \otimes U_B)(\Sigma_A \otimes \Sigma_B)(V_A \otimes V_B)^\top.$$

Hierbij werd gebruik gemaakt van de eigenschap dat de getransponeerde van het kroneckerproduct gelijk is aan het kroneckerproduct van de getransponeerdes. Vervolgens zijn $U_A \otimes U_B$ en $V_A \otimes V_B$ orthogonale matrices aangezien er geldt dat:

$$(U_A \otimes U_B)^\top (U_A \otimes U_B) = (U_A^\top \otimes U_B^\top)(U_A \otimes U_B) = (U_A^\top U_A) \otimes (U_B^\top U_B) = I \otimes I = I.$$

Bovendien is $\Sigma_A \otimes \Sigma_B$ een diagonaalmatrix. Uit de definitie van het kroneckerproduct volgt namelijk dat de blokmatrices $(\Sigma_A)_{ij}\Sigma_B$ enkel niet-nul zijn op de hoofddiagonaal ($i = j$), waar ze het product vormen van een scalaire waarde met de diagonaalmatrix Σ_B. De bekomen uitdrukking voor $A \otimes B$ is dus een geldige SVD. Dan geldt, door de definitie van de pseudo-inverse op de SVD:

$$(A \otimes B)^+ = (V_A \otimes V_B)(\Sigma_A \otimes \Sigma_B)^+ (U_A \otimes U_B)^\top.$$

Er is net bewezen dat $\Sigma_A \otimes \Sigma_B$ een diagonaalmatrix is. De pseudo-inverse van een diagonaalmatrix wordt berekend door de niet-nul diagonaalelementen te inverteren. Die bewerking komt voor die elementen dus overeen met de normale inverse. Hierdoor geldt de eigenschap $(X \otimes Y)^{-1} = X^{-1} \otimes Y^{-1}$ ook voor de pseudo-inverse van deze diagonaalmatrices. Door dit toe te passen, en daarna de mixed-product eigenschap, volgt dat

$$\begin{aligned}(A \otimes B)^+ &= (V_A \otimes V_B)(\Sigma_A^+ \otimes \Sigma_B^+)(U_A \otimes U_B)^\top \\ &= (V_A \Sigma_A^+ U_A^\top) \otimes (V_B \Sigma_B^+ U_B^\top) = A^+ \otimes B^+.\end{aligned}$$

Hiermee wordt

$$c = (A^+ \otimes B^+)\,\mathrm{vec}(F).$$

Door de identiteit

$$(X \otimes Y)\,\mathrm{vec}(M) = \mathrm{vec}(YMX^\top),$$

te gebruiken met $X = A^+$, $Y = B^+$ en $M = F$, volgt

$$c = \mathrm{vec}\Big(B^+ F (A^+)^\top\Big).$$

Dit is de gezochte uitdrukking.

Opgave 4

Voor de validatie van de implementatie is een test gebruikt waarvoor de exacte oplossing gekend is. Eerst werd een coëfficiëntenmatrix C_{true} gekozen. Daarna zijn de bijhorende data op het rooster opgebouwd via

$$F = B\, C_{\text{true}}\, A^\top.$$

Deze werkwijze volgt uit de stap $Z = BCA^\top$ in het bewijs van Opgave 1. Dit maakt dus de data op basis van de functiewaarden van de veelterm. Vervolgens is met de implementatie een schatting C_{est} berekend uit (x, y, F, n, m).

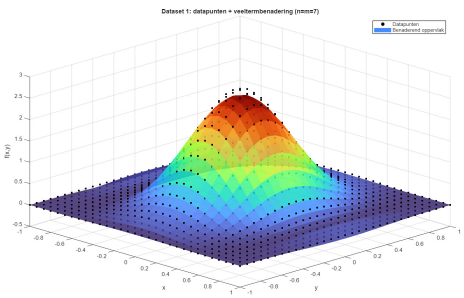
De correctheid is gecontroleerd met drie foutmaten:

$$\|C_{\text{est}} - C_{\text{true}}\|_\infty, \qquad \|C_{\text{est}} - C_{\text{true}}\|_F, \qquad \|B\, C_{\text{est}}\, A^\top - F\|_F.$$

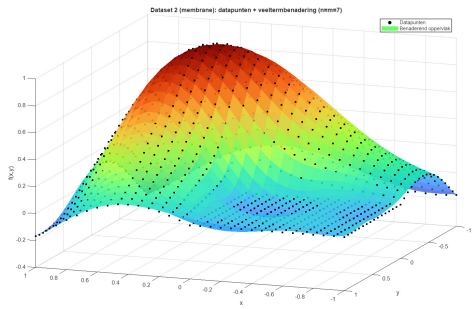
Alle maten liggen op machineprecisie. Dat bevestigt dat de opbouw van A en B, de matrixformule en het oplossen met de backslashoperator consistent zijn.

Opgave 5

De twee datasets op het 31×31-rooster zijn benaderd met maximale veeltermgraad $n = m = 7$. Het benaderende oppervlak met bijhorende matrix Z is berekend met $Z = BCA^\top$, aangezien in Opgave 1 aangetoond is dat dit equivalent is. Dit is eveneens efficiënter. De reden daarvoor is dat $\Gamma = A \otimes B$ een 961×64-matrix is, want A en B hebben dimensie 31×8. Bovendien wordt die grote matrix nadien nog vermenigvuldigd met de vector c om $z = \Gamma c$ te berekenen. Dit vereist meer rekenkracht en geheugen dan de eerste manier, want die vermenigvuldigt slechts matrices van dimensie 31×8 en 8×8.



Figuur 1.1: Veeltermbenadering ($n = m = 7$) van Dataset 1 (exponentieel).



Figuur 1.2: Veeltermbenadering ($n = m = 7$) van Dataset 2 (`membrane`).

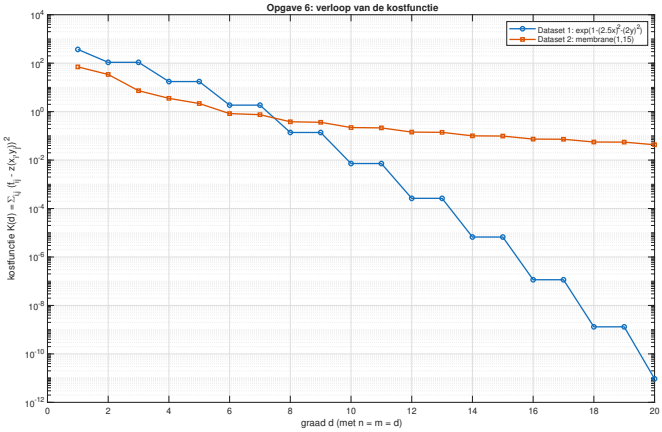
Figuur 1.1 en Figuur 1.2 tonen de veeltermoppervlakken en de datapunten. De gladde exponentiële functie (Dataset 1) wordt goed benaderd. De `membrane`-functie (Dataset 2) heeft echter grote afgeleiden nabij de randen van het L-vormige domein. Veeltermen zijn slecht voor het benaderen van dergelijke niet-gladde functies, wat leidt tot oscillaties aan de randen.

Opgave 6

Om de veeltermbenaderingen kwantitatief te evalueren, is het verloop van de kostfunctie berekend aan de hand van de formules uit Opgave 1:

$$K(d) = \sum_{i,j} (f_{ij} - z(x_i, y_j))^2 = \|F - Z\|_F^2 = \|\text{vec}(F) - \text{vec}(Z)\|_2^2.$$

De kost is berekend voor de graden $n = m = d$ gaande van 1 tot en met 20. Figuur 1.3 toont een semi-logaritmische plot van deze kost in functie van d.



Figuur 1.3: Kostfunctie van de bivariate veeltermbenadering met graad $n = m = d = 1, ..., 20$ voor de exponentiële functie $f_{ij} = \exp(1 - (2.5x_i)^2 - (2y_j)^2)$ (Dataset 1) en de `membrane`-functie (Dataset 2).

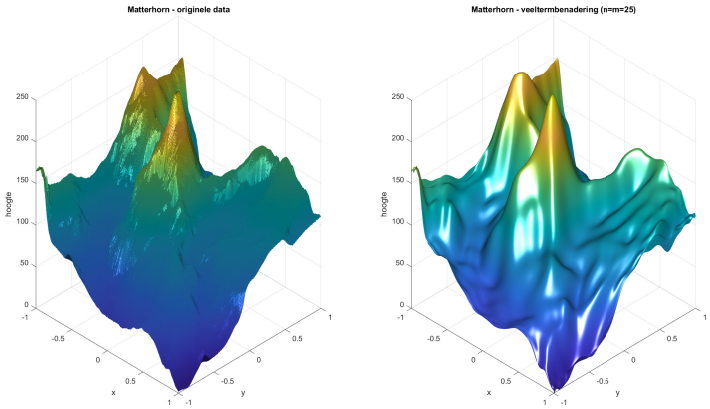
Figuur 1.3 toont dat de kostfunctie voor de gladde exponentiële functie (Dataset 1) snel en monotoon daalt richting de machineprecisie. Een bivariate veeltermbenadering is voor deze functie dus een geschikte benaderingsmethode.

Voor de `membrane`-functie (Dataset 2) stagneert de kostfunctie echter op een relatief hoge fout. Zelfs bij het verhogen van de graad tot $d = 20$ neemt de fout nog nauwelijks af. Dit bevestigt de oscillaties in Opgave 5. Zoals daar reeds uitgelegd is, is een veelterm niet geschikt om deze functie te benaderen vanwege het feit dat de eerste afgeleide groot is op de rand van het L-vormig domein. Het verhogen van de graad helpt dan niet om de fout te verkleinen, maar maakt de oscillaties erger over de rest van het domein.

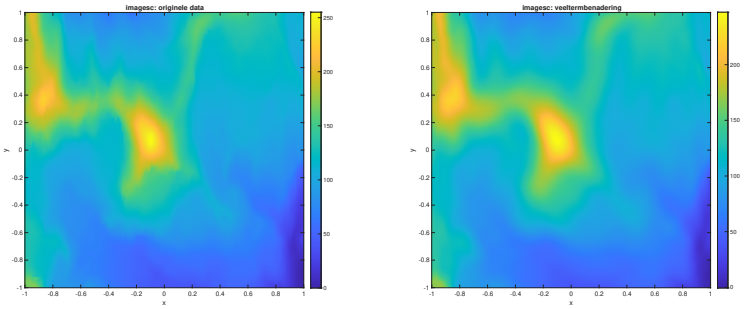
Om ten slotte de `membrane`-functie beter te benaderen, kunnen er tensorproduct-splinefuncties gebruikt worden. Voor twee dimensies en bv. $k = 3$, zijn dit dan bikubische splines. Deze kunnen geschreven worden als een lineaire combinatie van de tensorproducten van 1D-splines. Aangezien splines slechts in een aantal deelintervallen verschillend zijn van nul, is de invloed van elke coëfficiënt lokaal en zorgen de punten op de rand van het L-vormig domein niet meer voor benaderingsfouten in de rest van het domein.

Opgave 7

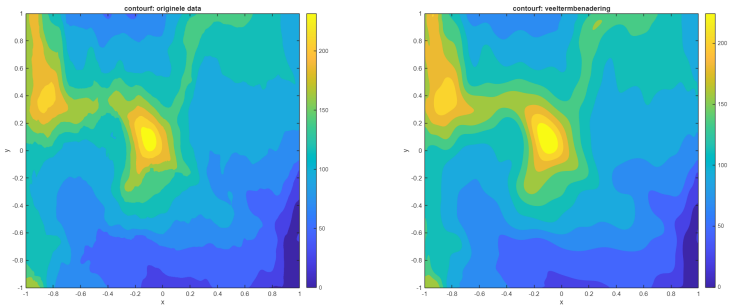
Figuur 1.4 toont de 3D-weergave van de Matterhorn-data en de berekende veeltermbenadering van graad $n = m = 25$. Vervolgens geven Figuur 1.5 en Figuur 1.6 respectievelijk het bovenaanzicht en de contourplots. In de buurt van de plaatsen waar er steile flanken zijn, is de benadering slechter. Bovendien worden de ruwe overgangen zachter.



Figuur 1.4: 3D-weergave van de Matterhorn-data (links) en de veeltermbenadering (rechts).



Figuur 1.5: Bovenaanzicht van de originele hoogtegegevens (links) en de veeltermbenadering (rechts).



Figuur 1.6: Contourplot van de oorspronkelijke data (links) en de veeltermbenadering (rechts).

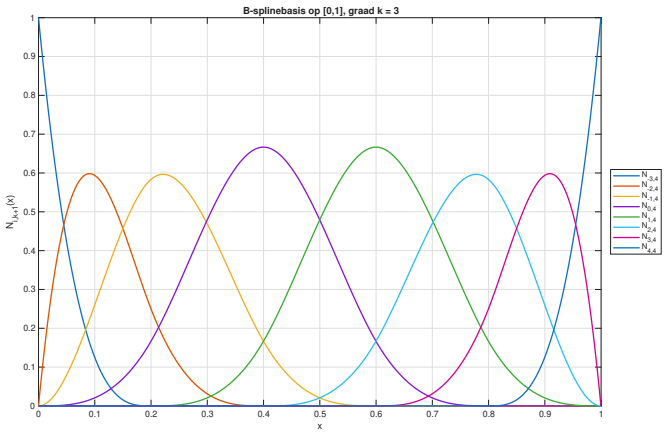
Opgave 8

Voor de visualisatie van de B-splinebasis op $[0, 1]$ is een kubische basis $(k = 3)$ gebruikt, en een open geklemde knopenrij met inwendig equistante knopen:

$$t = [\underbrace{0,\ 0,\ 0,\ 0}_{k+1},\ 0.2,\ 0.4,\ 0.6,\ 0.8,\ \underbrace{1,\ 1,\ 1,\ 1}_{k+1}].$$

Vanwege de $(k + 1)$-meervoudige knopen aan de rand, is de functiewaarde van alle basisfuncties gelijk aan nul buiten $[0, 1]$. Uit deze gegevens volgt dan het aantal basisfuncties $nb = \text{length}(t) - k - 1 = 8$. Elke basisfunctie $N_{i,4}$ is geëvalueerd via het algoritme van de Boor door als coëfficiëntenvector de i-de eenheidsvector $c = e_i$ te kiezen, zodat $s_i(x) = N_{i,4}(x)$.

De resultaten bevestigen de verwachte eigenschappen: niet-negativiteit, lokale steun en partitie van de eenheid $\sum_i N_{i,4}(x) = 1$ op $[0, 1]$ (numeriek tot op machineprecisie). Figuur 2.1 geeft de basisfuncties weer.



Figuur 2.1: Kubische B-splinebasis op $[0, 1]$ voor de gekozen knopenrij.

Opgave 9

De correctheid van `kkb_spline(t,x,f,xplot)` is gevalideerd met de volgende testen:

a) **Exacte reconstructietest (Test 1a):** data gegenereerd uit een gekende kubische spline met dezelfde knopenrij: de reconstructiefout ligt op machineprecisie.

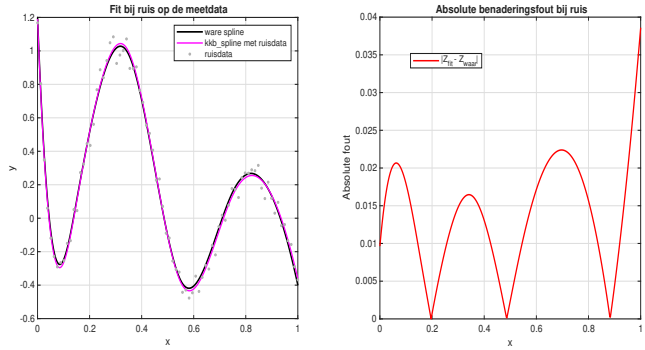
b) **Residu op trainingspunten (Test 1b):** evaluatie in de originele meetpunten met residumaten $\|r\|_\infty$, $\|r\|_2$ en RMSE: de fout ligt op machineprecisie.

c) **Robuustheid bij ruis (Test 2):** additieve ruis op de meetdata en vergelijking van de fit met de onderliggende trend.

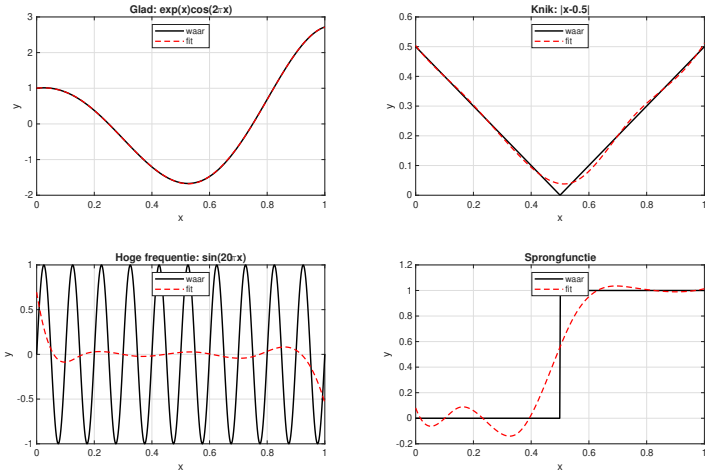
d) **Functie-afhankelijkheid (Test 3):** vergelijking op representatieve randgevallen (glad, knik, hoge frequentie, sprong).

e) **Randpunttest (Test 4):** expliciete controle van de evaluatie in $x = 0$ en $x = 1$.

Figuur 2.2 en Figuur 2.3 illustreren dit visueel voor respectievelijk Test 2 en 3. Figuur 2.2 toont dat de benadering stabiel blijft onder ruis. De fout is orde 10^{-2}. Figuur 2.3 toont dat gladde functies merkbaar beter benaderd worden dan niet-gladde of sterk oscillatorische functies.



Figuur 2.2: Splinebenadering bij ruis op de meetdata (links) en de absolute benaderingsfout (rechts) (Test 2).

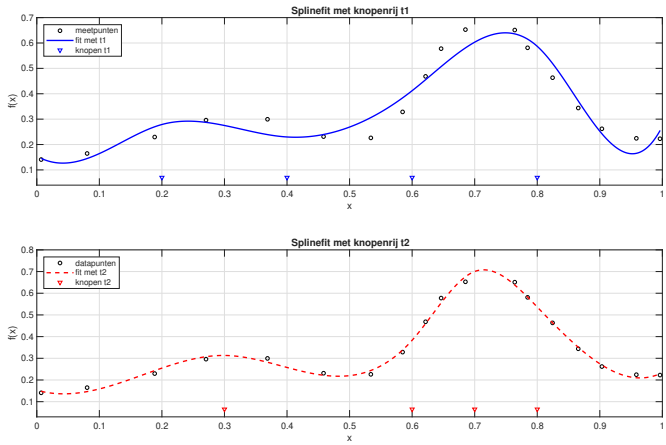


Figuur 2.3: Randgevallen voor functie-afhankelijkheid: glad, knik, hoge frequentie en sprongfunctie (Test 3).

Opgave 10

De meetgegevens uit `exercise10.mat` zijn benaderd met dezelfde splineprocedure als in Opgave 9, met knopenrijen t_1 en t_2. Figuur 2.4 geeft de vergelijking van beide benaderingen weer.

Op basis van de bekomen foutmaten op de meetpunten (RMSE en maximale absolute fout) levert t_2 de beste benadering. Dit is consistent met de knoopplaatsing: t_2 voorziet meer lokale flexibiliteit in het interval waar de data de sterkste vormverandering vertoont, terwijl t_1 de knopen equidistant verdeelt. In Figuur 2.4 zijn de knopen expliciet als markeringen op de x-as weergegeven.



Figuur 2.4: Vergelijking van de splinefits met knopenrijen t_1 en t_2.

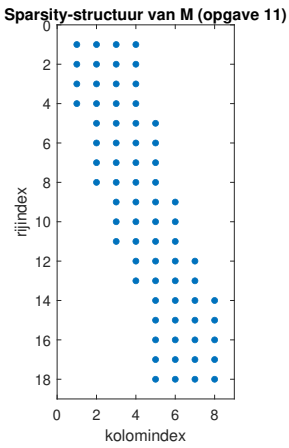
Opgave 11

Voor de gekozen knopenrij is de structuur van matrix M uit

$$Mc = f$$

gevisualiseerd met `spy`, zie Figuur 2.5. De matrix is schaars en bandachtig: per rij zijn slechts $k+1$ basisfuncties niet nul (voor kubische splines: maximaal 4). Dit volgt uit de lokale steun van B-splines; aan de randen ontstaan bijkomende structurele nullen.

Voor deze matrixstructuur is een **sparse Householder-QR-factorisatie** een efficiënte keuze. De matrix M is overgedetermineerd en sterk rechthoekig: het aantal rijen R is aanzienlijk groter dan het aantal kolommen $(n+k)$, zodat orthogonale Householder-transformaties bijzonder geschikt zijn voor een stabiele kleinstekwadratenoplossing. Householder-reflecties zijn numeriek stabiel. Een dichte QR-factorisatie negeert deze sparsiteit en is daarom minder efficiënt in tijd en geheugen.



Figuur 2.5: Sparsity-structuur van matrix M (via `spy`).

Opgave 12

In de praktijk hebben B-splines duidelijke voordelen ten opzichte van afgeknotte machtsfuncties als basis voor splinebenadering.

Ten eerste hebben B-splines *compacte lokale steun*: elke basisfunctie is slechts verschillend van nul op een beperkt aantal opeenvolgende knoopintervallen. Daardoor veroorzaakt een lokale wijziging van een coëfficiënt enkel een lokale wijziging van de benadering. Afgeknotte machtsfuncties $(x - \tau)_+^p$ zijn weliswaar afgeknot, maar binnen het domein $[0, 1]$ zijn ze enkel actief op $[\tau, 1]$. Hun invloed is dus minder lokaal en strekt zich typisch over een groter deel van het domein uit, wat het moeilijker maakt om de benadering lokaal te controleren.

Ten tweede is de numerieke conditionering van B-splinebases doorgaans aanzienlijk beter. De bijhorende stelsels zijn daardoor stabieler op te lossen, terwijl bases met afgeknotte machtsfuncties sneller slecht geconditioneerd raken, vooral bij hogere graad of wanneer knopen dichter op elkaar liggen.

Ten derde ontstaat bij B-splines een schaarse en bandachtige systeemmatrix, omdat in elk punt slechts een beperkt aantal basisfuncties niet nul is. Dit laat efficiënte algoritmen toe (zoals sparse QR), met lagere reken- en geheugenkost. Bij afgeknotte machtsfuncties is de matrix in het algemeen dichter en computationeel minder gunstig.

Tot slot kunnen B-splines efficiënt en stabiel geëvalueerd worden via het algoritme van de Boor, en zijn gladheidsvoorwaarden op knopen op een natuurlijke manier ingebouwd via de knoopstructuur en multipliciteiten. Deze combinatie van lokale controle, numerieke stabiliteit en rekenefficiëntie verklaart waarom B-splines de standaardkeuze zijn in praktische splinebenadering.